Proiect Rezolvarea Problemelor și Căutare - seriile 23 și 25

1 Algoritmi de Căutare Informată

1.1 Pancake Sorting Problem (3 puncte)

Se consideră o stivă formată din N clătite, fiecare având o dimensiune *distinctă*, numerotată cu valori întregi din intervalul $[1, N]$. Scopul este de a ordona clătitele *descrescător de jos în sus* (echivalent: cea mai mare clătită la bază, iar cea mai mică în vârf). Asupra stivei este permisă o singură operație, notată *flip(k)*, definită astfel:

> *flip(k)* inversează ordinea primelor k clătite din vârful stivei, adică realizează operația de *reverse* asupra primelor k elemente.

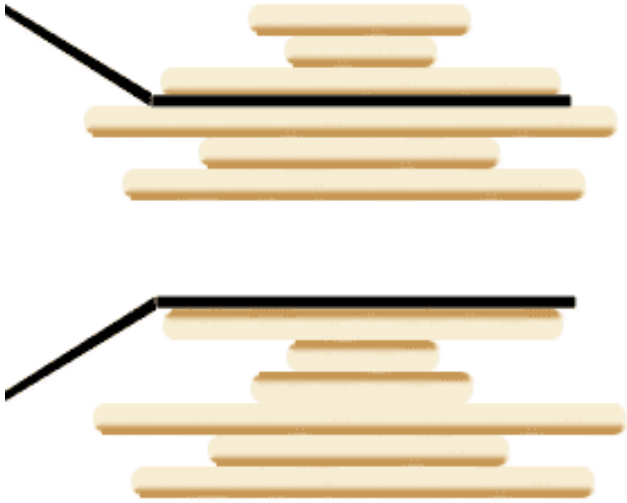


Figure 1: Operația *flip(k)* (sursă: Wikipedia)

Pornind de la o configurație inițială arbitrară a stivei, rezolvați următoarele cerințe:

1. Propuneți o euristică *admisibilă* și cât mai *informativă* pentru această problemă. Justificați de ce euristica propusă este admisibilă. (1p)
2. Implementați algoritmul $A*$ folosind euristica definită la punctul anterior. Soluțiile care folosesc o euristică inadmisibilă nu vor fi punctate. (1p)
3. Implementați algoritmul *IDA** folosind aceeași euristică. Soluțiile care folosesc o euristică inadmisibilă nu vor fi punctate. (1p)

Explicația pentru subpunctul 1 poate fi inclusă într-o celulă Markdown din notebook-ul `.ipynb`, împreună cu restul rezolvării, sau poate fi predată separat într-un fișier PDF.
Implementarea poate fi trimisă fie sub forma unui notebook `.ipynb`, fie sub forma a două fișiere `.py` (unul pentru punctul 2 și unul pentru punctul 3).

2 Algoritmi de Căutare Adversarială

2.1 Othello (2 puncte)

Othello este un joc strategic pentru doi jucători, desfășurat pe o tablă de dimensiune 8×8. Fiecare jucător controlează piese de o culoare: *negru* sau *alb*. La începutul jocului, pe cele patru poziții centrale ale tablei se află 4 piese, dispuse astfel: două piese negre și două piese albe, așezate pe diagonală.Regulile jocului sunt următoarele:

- Jucătorii mută alternativ, primul jucător fiind cel care controlează piesele negre.
- O mutare constă în plasarea unei piese proprii pe o celulă liberă a tablei.
- O mutare este *validă* doar dacă piesa plasată încadrează cel puțin o piesă a adversarului între ea și o altă piesă proprie, pe una dintre cele 8 direcții posibile: orizontal, vertical sau diagonal.
- Toate piesele adversarului încadrate în acest mod sunt *întoarse* și devin piese ale jucătorului curent.
- Dacă un jucător nu are nicio mutare validă, acesta pasează, iar adversarul mută din nou.
- Jocul se încheie atunci când tabla este complet ocupată sau când niciunul dintre cei doi jucători nu mai are mutări valide.
- Câștigă jucătorul care are cele mai multe piese pe tablă la finalul jocului.

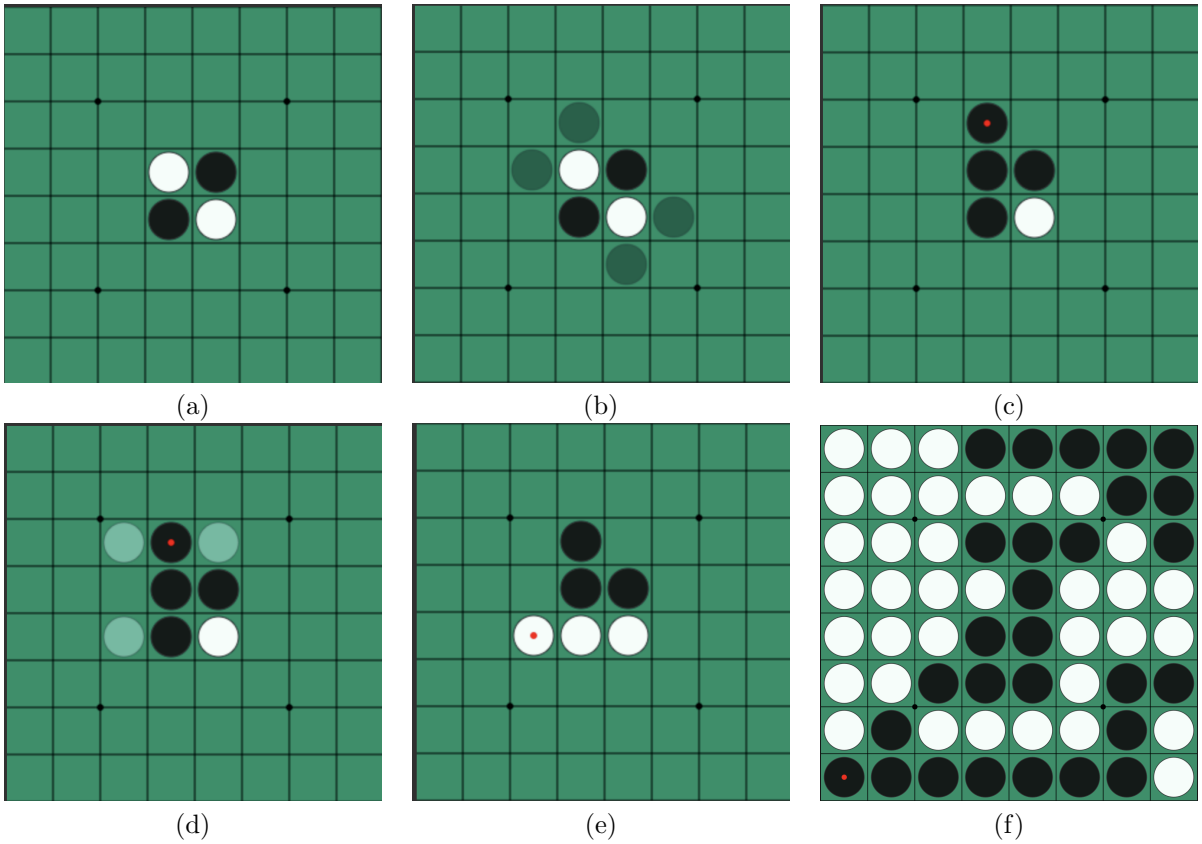


Figure 2: (a) Starea inițială a jocului; (b) Jucătorul cu piesele negre are patru mutări valide în starea curentă; (c) Mutare realizată de jucătorul cu piesele negre; (d) Jucătorul cu piesele albe are trei mutări valide în starea curentă; (e) Mutare realizată de jucătorul cu piesele albe; (f) Stare finală a jocului, în care jucătorul cu piesele albe câștigă cu scorul 36–28.

Rezolvați următoarele cerințe:

1. Definiți o funcție euristică de evaluare $e : S \to \mathbb{R}$ pentru stările neterminale, care să estimeze avantajul jucătorului 1 într-o stare s. Justificați alegerea făcută. (1p)
2. Implementați algoritmul **Minimax** cu **Alpha-Beta Pruning**, considerând o adâncime de căutare limitată la un parametru configurabil $d_{\max}$. (1p)

Pentru definirea unei funcții de evaluare cât mai bune, puteți consulta și următoarele lucrări:

- Sannidhanam V. & Annamalai M. – *An Analysis of Heuristics in Othello*, link
- *IAGo vs Othello* (2012), CEUR Workshop Proceedings, vol. 1107, link
- Korf R. – *Minimax Search: Othello Results*, link

Aceste materiale pot fi utile în alegerea și justificarea unei funcții euristice de evaluare.

2.2 Hog (1 punct)

Hog este o variantă mai *explozivă* a jocului clasic cu zaruri *Pig*, jucat de doi jucători. Scopul jocului este acumularea a cel puțin 100 de puncte. În joc se folosesc zaruri clasice, fiecare având 6 fețe numerotate de la 1 la 6, fiecare rezultat fiind echiprobabil. Regulile jocului sunt următoarele:

- La începutul fiecărei ture, jucătorul decide câte zaruri să arunce, alegând un număr $n \in [0, 10]$.
- *Free Bacon.* Dacă jucătorul alege $n = 0$, nu aruncă niciun zar și primește automat

  $$1 + \max(\text{cifra unităților scorului adversarului},\ \text{cifra zecilor scorului adversarului})$$

  puncte.
- Dacă $n > 0$, jucătorul aruncă cele n zaruri.
- *Pig Out.* Dacă oricare dintre cele n zaruri aruncate are valoarea 1, jucătorul primește doar *1 punct* în acea tură.
- Dacă niciun zar nu arată valoarea 1, jucătorul câștigă

  $$\sum_{i=1}^{n} zar_i$$

  puncte, adică suma valorilor obținute la aruncarea zarurilor.

Jucătorii alternează turele, iar primul jucător care ajunge sau depășește *100 de puncte* câștigă jocul. Rezolvați următoarele cerințe:

1. Propuneți o funcție de evaluare potrivită pentru stările jocului și justificați alegerea făcută. (0.5p)
2. Implementați algoritmul *Expectimax* folosind funcția de evaluare definită la punctul anterior. (0.5p)

3 Algoritmi Genetici

3.1 Crearea labirintului (3 puncte)

Să se implementeze un algoritm genetic care să genereze un labirint de dimensiune N×M. Labirintul va conține obiecte colecționabile, pericole și zone ce pot fi accesate doar cu anumite chei. Se va considera că personajul care se deplasează în labirint are energie limitată și pierde (inițial) 1p energie la fiecare pas.

Exemple de **obiecte colecționabile**:

1. chei care permit accesul în anumite celule. Chiar dacă aceste celule nu sunt înconjurate de pereți, personajul nu poate pătrunde fără o cheie cu identificator egal cu al zonei blocate;
2. mâncare care crește energia personajului cu M_i (poate fi diferit pentru diferitele unități de mâncare);
3. scuturi care permit personajului să intre în celula unui pericol fără a-l afecta. Se scade în mod automat un scut din numărul de scuturi deținute de personaj, când personajul iese din celulă.

Exemple de **pericole**:

1. monștri care ocupă o celulă de labirint. Personajul se poate lupta cu ei dar o să îl conste K_i energie, unde K_i depinde de monstru. Odată învins monstrul, acesta dispare din labirint;
2. mâncare otrăvita. Personajului îi crește energia inițial cu M_i, dar din acel moment crește cu 1 și cantitatea de energie consumată la fiecare pas.

După generarea labirintului, utilizatorul va putea chiar să îl parcurgă și să încerce să rezolve problema. Se va afișa la fiecare pas energia personajului și inventarul.

Funcția de evaluare (fitness) a design-ului va avea o valoare mai mare cu cât e mai complex labirintul. Trebuie să țină cont de următorii parametri:

a) ND - numărul total de drumuri dintre start și ieșiri. Dacă $ND = 0$, fitness-ul primește o penalizare maximă. Dar cu cât ND e mai mare, cu atât scorul de complexitate trebuie să scadă;
b) posibilitatea de a finaliza drumul înainte de a pierde toată energia (are suficientă mâncare în labirint sau drumul parcurs până la ieșire se încadrează în energia inițială);
c) LS - lungimea soluției optime trebuie să fie cât mai mare;
d) NC - numărul de cotituri din soluția optimă trebuie să fie cât mai mare;
e) Nivelul de explorare. Obiectele de colectat trebuie să fie plasate în locații accesibile, dar pe cât posibil nu direct pe drumul cel mai scurt către ieșire, încurajând o explorare mai profundă;
f) Procentajul de celule pereți (PC) trebuie să fie aproximativ $20 - 40\%$, pentru a avea o structură de labirint suficient de complexă;
g) NPI - Numărul de pereți înconjurați de alți pereți în toate părțile (sus, jos, stânga, dreapta); aria zonelor complet izolate de personaj să fie minimă.
h) NP - numărul de pericole să fie între $5 - 15\%$ dar e preferabil să fie cât mai mare.

Exemplu de funcție de fitness pentru evaluarea complexității (observați că am asociat anumite ponderi unor parametri):

$$f(ND, LS, NC, NPI, NP) = \frac{NP * NC}{ND} + 0.3 * LS - 1.5 * NPI$$

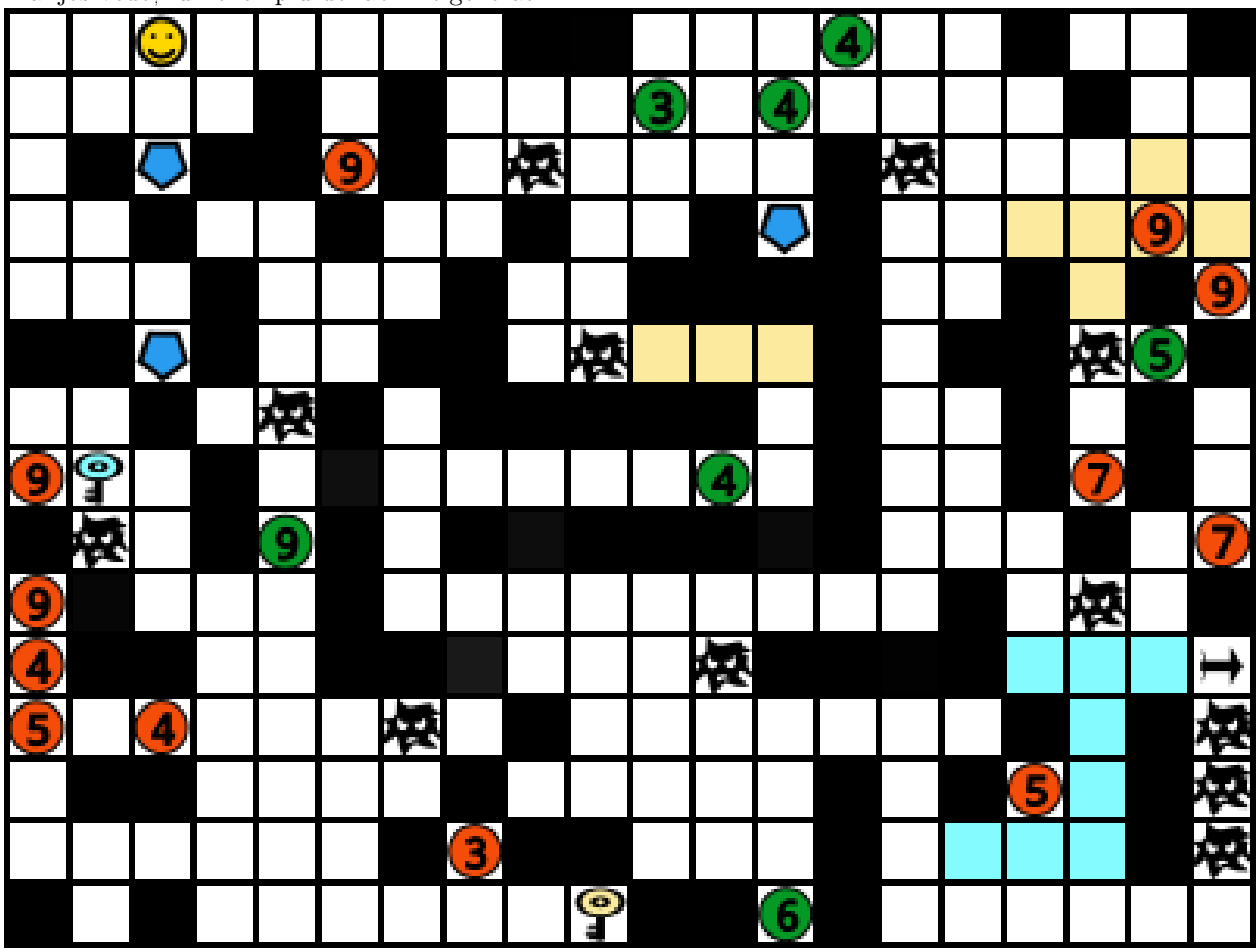
Cerințe:

1. (0.5p) Pentru populația inițială veți folosi un DFS randomizat (cromozomii deja să fie labirinturi valide): `https://en.wikipedia.org/wiki/Maze_generation_algorithm#Recursive_implementation`;
2. (0.5p) Adăugați încă un tip de obiect colecționabil și de pericol care să influențeze drumul personajului. Încărcați în program, dintr-un fișier de intrare diversele setări necesare pentru labirint, cum ar fi energia inițială a personajului, valori minime și maxime pentru diverșii parametri aleatori care descriu labirintul etc.
3. (1p) Găsiți (și implementați) o funcție de fitness, alta decât în exemplu, care să cuprindă cât mai mulți parametri, adăugând minim 2 parametri în plus (incluzând obiectele colecționabile și pericolele

adăugate de voi). Atenție la parametrii care influențează pozitiv vs. cei care influențează negativ complexitatea labirintului;

4. (1p) Implementați, folosind un algoritm genetic crearea labirintului. Cromozomii vor fi matrice care conțin datele labirintului. Alegeți voi strategiile de selecție, încrucișare și mutație cele mai potrivite pentru acest tip de problemă (la prezentare veți explica de ce le-ați ales).

Mai jos vedeți un exemplu de labirint generat:



Legenda:

1. celulele albe sunt zone libere fără restricții, iar celulele negre sunt obstacole;
2. figura zâmbitoare (linia 0, coloana 2) indică pozitia de start;
3. săgeata (linia 10, coloana 14) indică ieșirea din labirint (poziția finală);
4. cercurile verzi reprezintă mâncare otrăvită iar cele portocalii reprezintă mâncare obișnuită (în cercuri se vede și câtă energie oferă mâncarea);
5. celulele cu pentagon albastru reprezintă scuturile;
6. în exemplu există o cheie galbenă și una de culoare albastru deschis pentru a debloca zonele cu aceleași culori;
7. monstruleții sunt petele negre furioase (nu s-a mai afișat și energia necesară învingerii acestora).